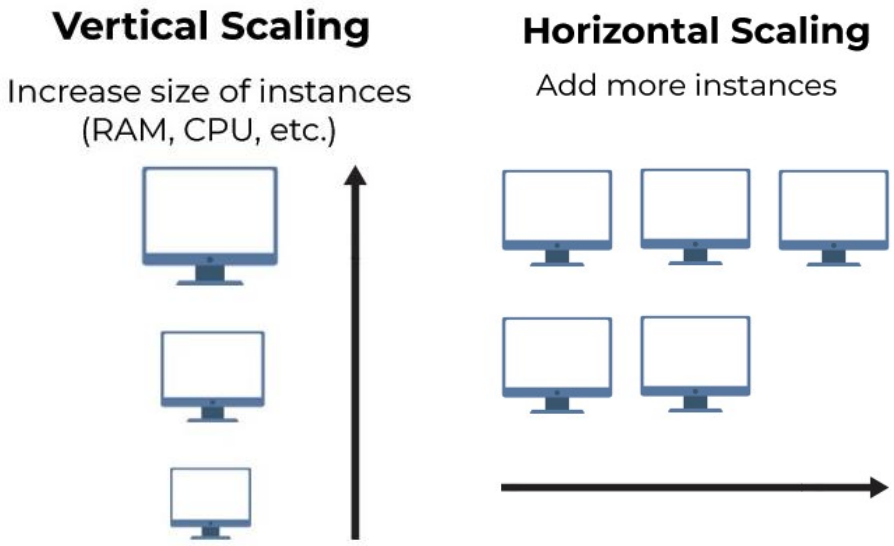
Data Replication

Distributed Data: Reasons for Horizontal Scaling

- *Scalability*
  - Data volume
  - Read load
  - Write load
- *Fault tolerance/high availability*
  - Multiple machines give redundancy
- *Latency*
  - Users around the world
  - Servers at various locations worldwide
  - Each user can be served from a datacenter that is geographically close to them
  - Avoids to wait for network packets to travel halfway around the world

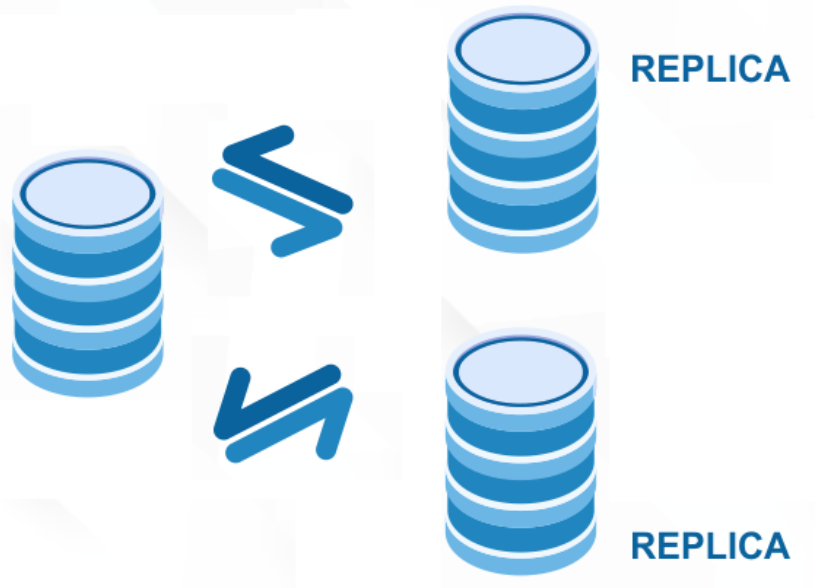


Replication

- Keeping a copy of the same data on multiple machines that are connected via a network
- This lecture's assumption:
  - Small dataset
  - Each machine can hold a copy of the entire dataset

Replication

- Replication is easy
  - If the data does not change over time
  - Just copy the data to every node once
- Replication is difficult
  - Lies in ***handling changes*** to replicated data
- Replica
  - Each node that stores a copy of the database
- Every <u>write</u> to the database needs to be processed by every replica



Leader-Based Replication

- The most common solution for this:
  - Leader-based replication
  - Also known as active/passive or master–slave replication
  - One leader
  - Multiple followers

Leader-Based Replication

- When clients want to write to the database:
  - They must send their requests to the leader
    - The leader first writes the new data to its local storage

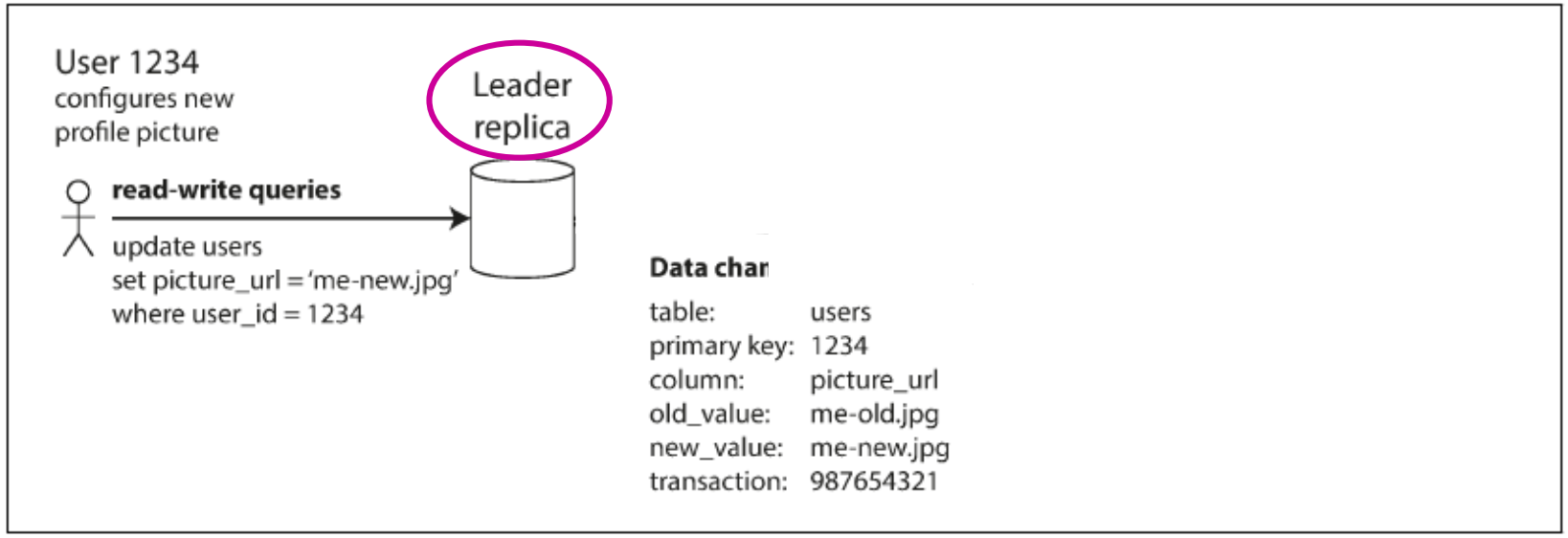


Figure 5-1. Leader-based (master–slave) replication.

Leader-Based Replication

- Whenever the leader writes new data to its local storage:
  - It also sends the data change to all its followers as part of a replication log or change stream

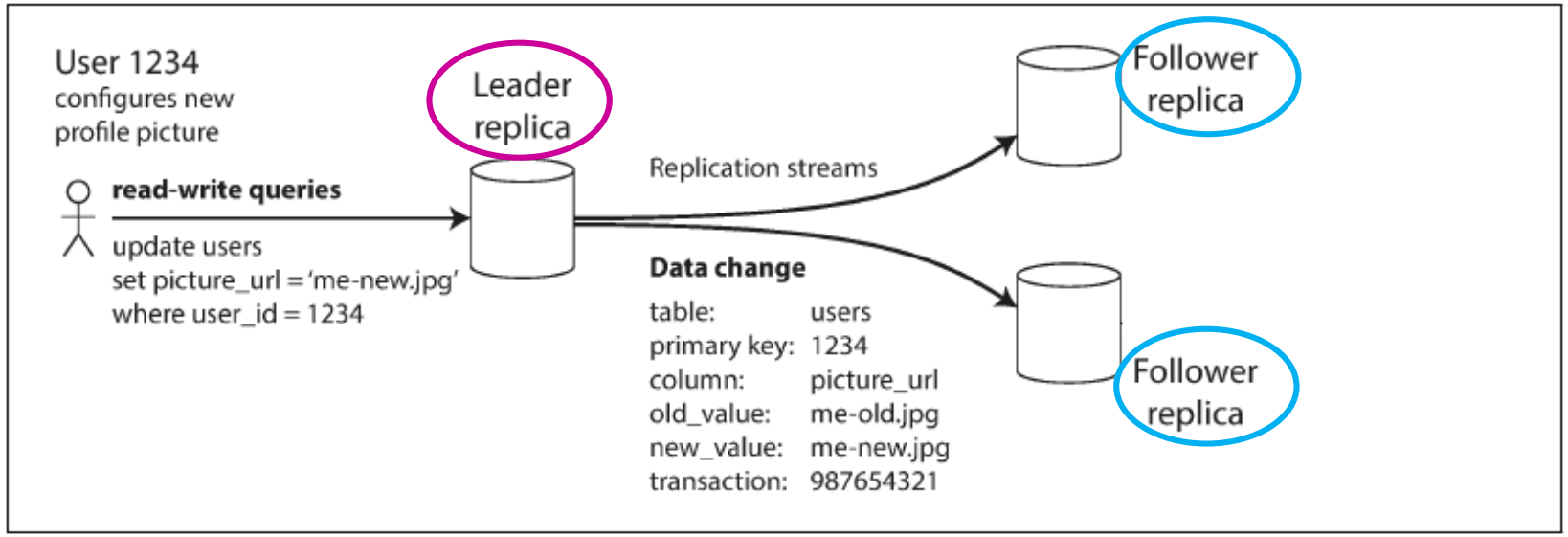


Figure 5-1. Leader-based (master–slave) replication.

Leader-Based Replication

- Each follower
  - takes the log from the leader
  - updates its local copy of the database accordingly

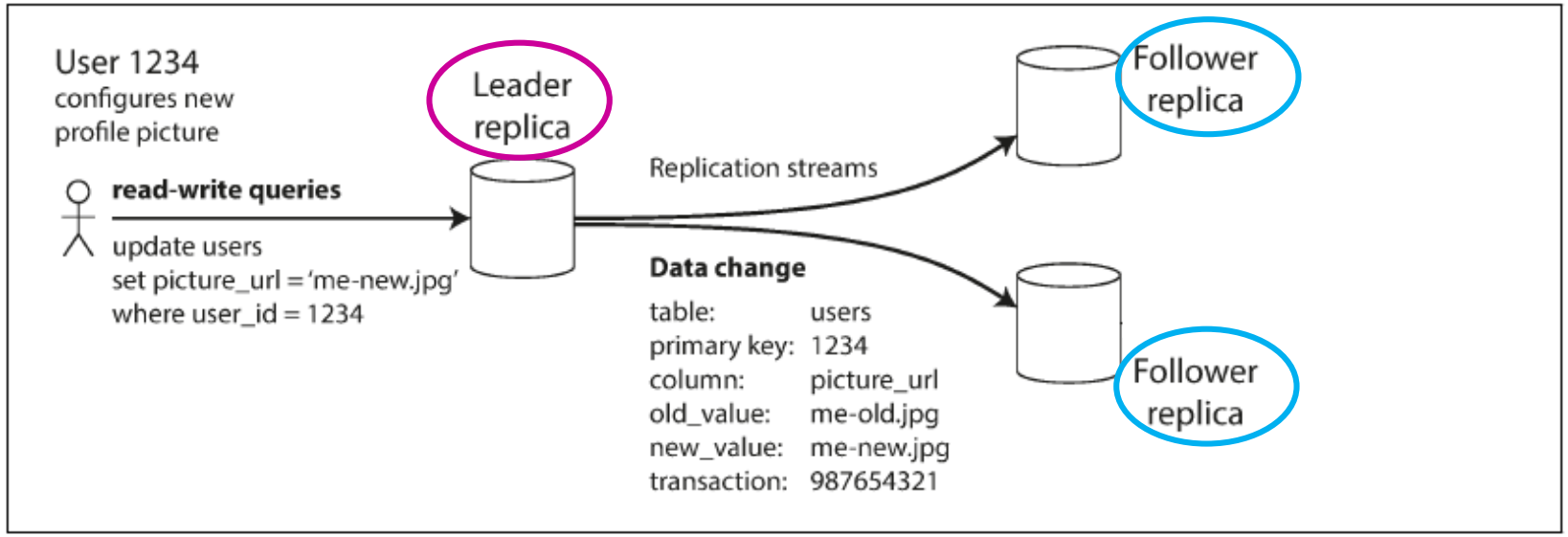


Figure 5-1. Leader-based (master–slave) replication.

Leader-Based Replication

- When a client wants to read from the database, it can query either the leader or any of the followers.

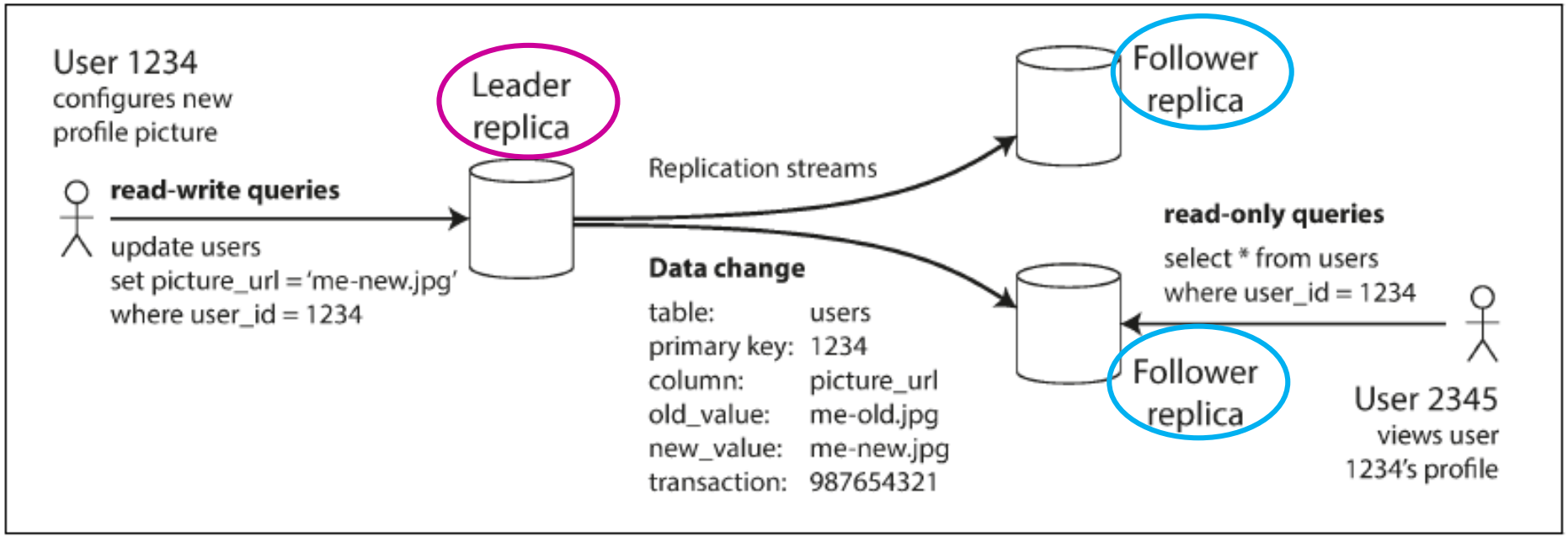


Figure 5-1. Leader-based (master–slave) replication.

Leader-Based Replication

- Writes are only accepted on the leader
- Reads: the leader or any of the followers
- The followers are read-only from the client's point of view

Leader-Based Replication

- A built-in feature of many relational databases:
  - PostgreSQL (since version 9.0)
  - MySQL
  - Oracle Data Guard
  - SQL Server’s AlwaysOn Availability Groups
- Also used in some nonrelational databases:
  - MongoDB
  - RethinkDB
  - Espresso

Synchronous Versus Asynchronous Replication

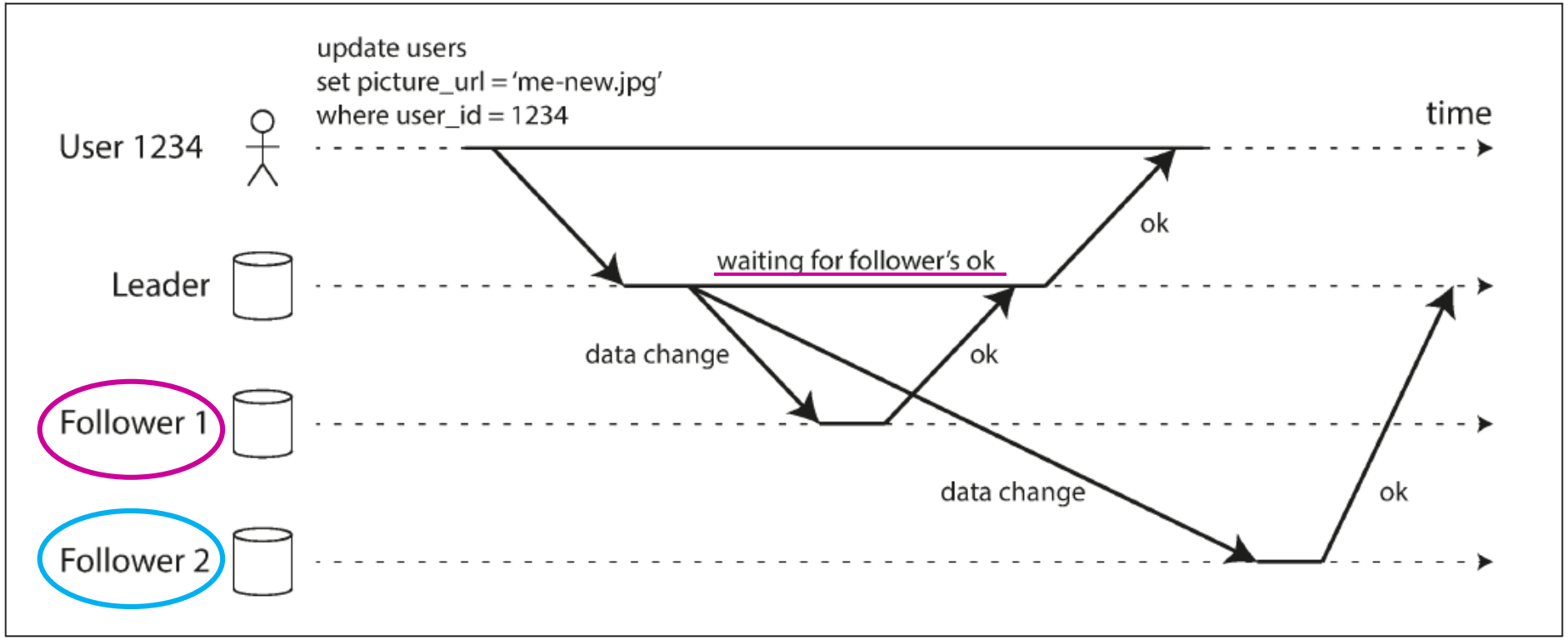
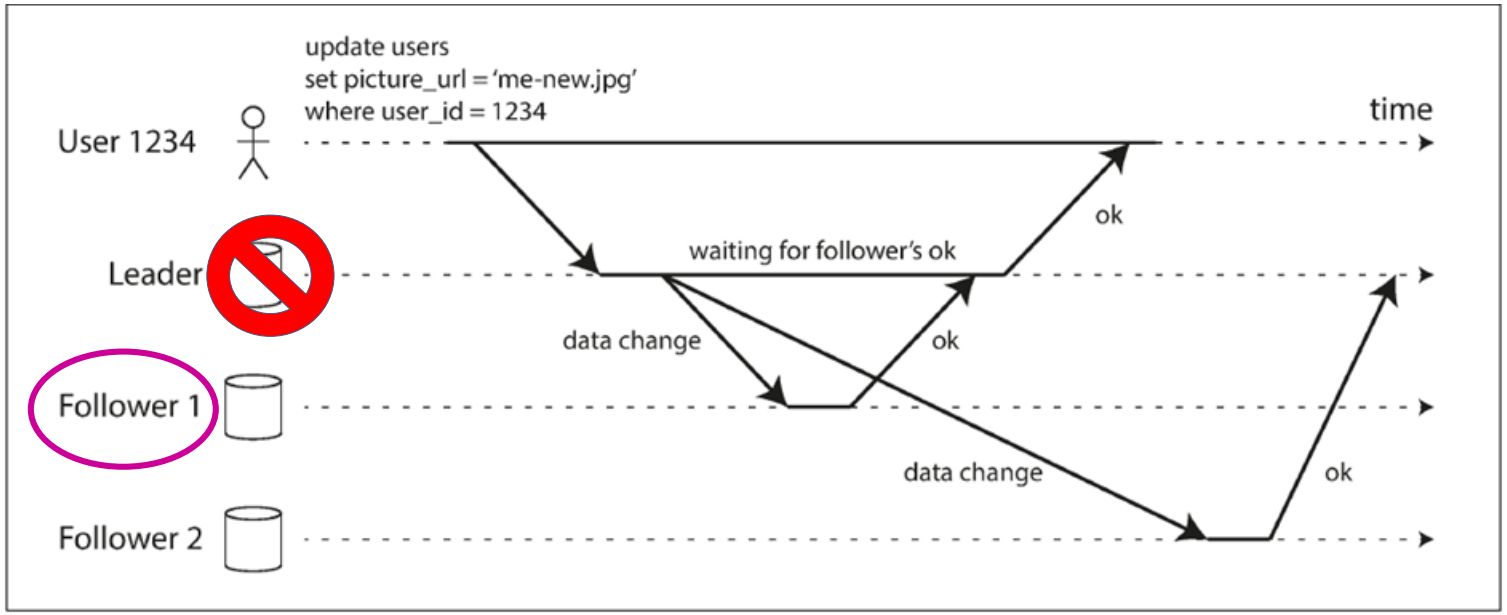


Figure 5-2. Leader-based replication with one synchronous and one asynchronous follower.

Advantages of Synchronous Replication

- The follower is guaranteed to have an up-to-date copy of the data that is consistent with the leader.
- If the leader suddenly fails, we can be sure that the data is still available on the follower.



The Disadvantage of Synchronous Replication

- If the synchronous follower doesn't respond, the write cannot be processed.
- The leader must
  - block all writes
  - wait until the synchronous replica is available again

Synchronous Replication

- Impractical for all followers to be synchronous
- In practice, if you enable synchronous replication on a database:
  - one of the followers is synchronous
  - the others are asynchronous
- This configuration is sometimes also called semi-synchronous
  - Guarantees that you have an up-to-date copy of the data on at least two nodes:
    - The leader
    - One synchronous follower

Asynchronous Replication

- Often, leader-based replication is configured to be completely asynchronous.
- If the leader fails and is not recoverable:
  - Any writes that have not yet been replicated to followers are lost. (-)
  - A write is not guaranteed to be durable, even if it has been confirmed to the client. (-)
- However, the leader can continue processing writes, even if all its followers have fallen behind. (+)

Setting Up New Followers

- Copying data files from one node to another is not sufficient:
  - Clients are constantly writing to the database
  - The data is always in flux
- Locking the database (making it unavailable for writes)?
  - Go against our goal of high availability.

Setting Up New Followers

1. Take a consistent snapshot of the leader's database at some point in time, without taking a lock on the entire database
2. Copy the snapshot to the new follower node
3. The follower connects to the leader and requests all the data changes that have happened since the snapshot was taken
4. When the follower has processed the backlog of data changes since the snapshot, we say it has caught up.
   - It can now continue to process data changes from the leader as they happen.

Handling Node Outages

- Any node in the system can go down (due to a fault, planned maintenance)
- Our goal is to keep:
  - the system as a whole running despite individual node failures
  - the impact of a node outage as small as possible

How Do You Achieve High Availability with Leader-Based Replication?

- Follower failure: Catch-up recovery
  - Each follower keeps a log of the data changes (on its local disk) it has received from the leader
  - The follower can recover quite easily from the log
  - The follower can connect to the leader and request all the data changes that occurred during the time when the follower was disconnected

How Do You Achieve High Availability with Leader-Based Replication?

- Leader failure: Failover
  - One of the follower needs to be promoted to the **new leader**
  - Client needs to be reconfigured to send writes to the new leader
  - Other followers start consuming data changes from new leader
  - Failover can happen manually or automatically
  - Automatic failover process:
    1. Determining that leader has failed - using timeouts
    2. Choosing a new leader – Consensus problem – Most up-to-date replica
    3. Reconfiguring the system to use the new leader

Implementation of Replication Logs

- Statement-based replication
  - Simplest case
  - The leader
    - logs every write request (statement) that it executes
    - sends that statement log to its followers
  - Ex: Every INSERT, UPDATE, or DELETE statement is forwarded to followers, and each follower parses and executes that SQL statement as if it had been received from a client.

Implementation of Replication Logs

- Statement-based replication
  - Problems?
    - Nondeterministic functions such as: NOW(), RAND()
      - Likely to generate a different value on each replica
    - If statements use an autoincrementing column?
    - If they depend on the existing data in the database (e.g., UPDATE … WHERE <some condition>)?
      - They must be executed in exactly the same order on each replica, or else they may have a different effect.

Implementation of Replication Logs

- Statement-based replication
  - Was used in MySQL before version 5.1
  - It is still sometimes used today, as it is quite compact
  - By default, MySQL now switches to row-based replication if there is any nondeterminism in a statement

Implementation of Replication Logs

- Logical (row-based) log replication
  - Send row level key and data
  - *A logical log contains* for a relational database:
    - For an inserted row: the new values of all columns
    - For a deleted row: enough information to uniquely identify the row that was deleted – primary key / old values of all columns
    - For an updated row: enough information to uniquely identify the updated row, and the new values of all columns
  - Allows the leader and the follower to run different versions of the database software, or even different storage engines.

Implementation of Replication Logs

- Write-ahead log (WAL) shipping
  - Besides writing the log to disk, the leader also sends it across the network to its followers (detailed physical write change log).
  - When the follower processes this log, it builds a copy of the exact same data structures as found on the leader.
  - Is used in: PostgreSQL and Oracle

Implementation of Replication Logs

- Write-ahead log (WAL) shipping
  - Main disadvantage:
    - The log describes the data on a very low level:
      - which bytes were changed in which disk blocks
    - Closely coupled to the storage engine.
    - Typically, not possible to run different versions of the database software on the leader and the followers.

Implementation of Replication Logs

- The replication approaches described so far:
  - Implemented by the database system.
- Trigger-based replication
  - Trigger: “Event-Condition-Action Rules”
  - A trigger lets you register custom application code that is automatically executed when a data change occurs in a database system.
  - Manually coded.

Implementation of Replication Logs

- Trigger-based replication
  - Has greater overheads than other replication methods (-)
  - More prone to bugs and limitations than the database's built-in replication (-)
  - More flexible (+)
    - Ex: Only replicate a subset of the data

Problems with Replication Lag

- Leader-based replication:
  - all writes to go through a single node
  - read-only queries can go to any replica
- For workloads that consist of mostly reads and only a small percentage of writes (a common pattern on the web):
  - Create many followers
  - Distribute the read requests across those followers
- You can increase the capacity for serving read-only requests simply by adding more followers.

Problems with Replication Lag

- Reading from an asynchronous follower:
  - May result in outdated information
- Inconsistencies in the database: same query with different results
  - on the leader
  - a follower at the same time
- Why? because not all writes have been reflected in the follower
- This inconsistency is just a temporary state
  - If you stop writing to the database and wait a while, the followers will eventually catch up and become consistent with the leader.

Problems with Replication Lag

- Eventual Consistency
  - *Replication lag* can be seconds or minutes
- Reading Your Own Writes: Figure 5-3

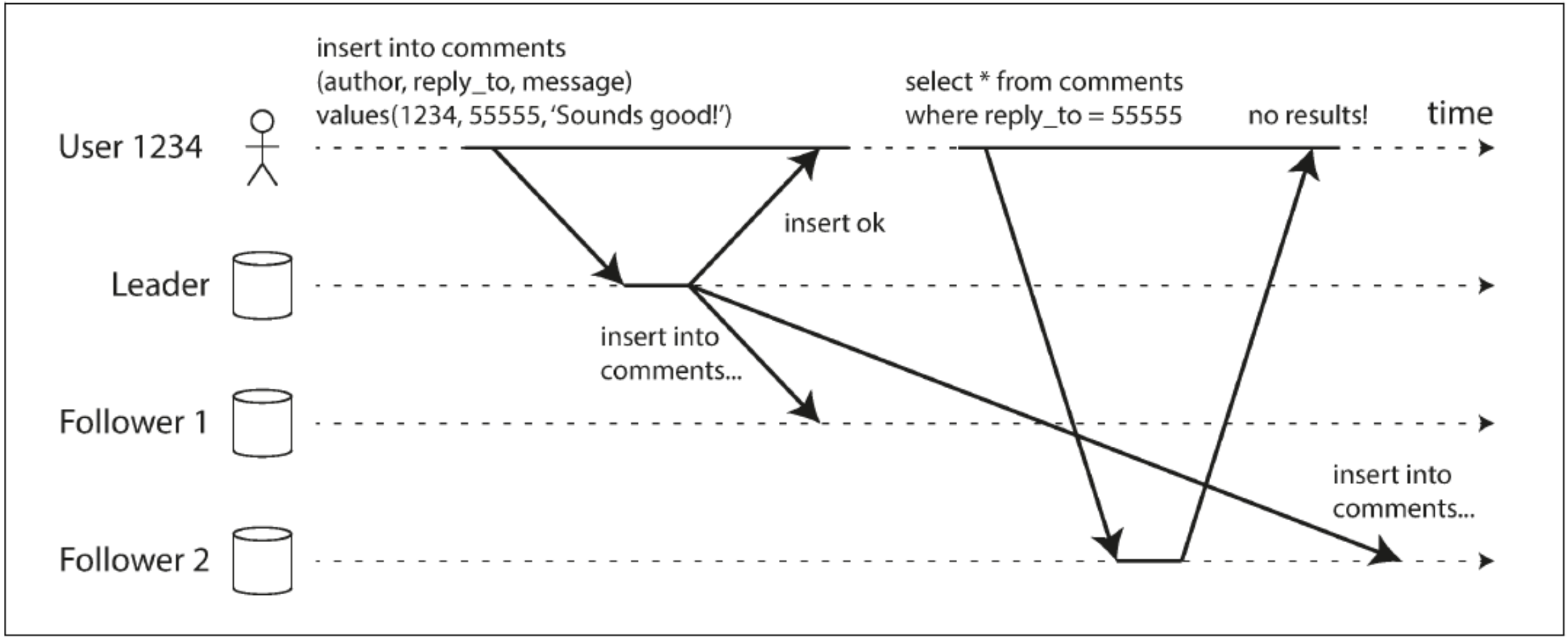


Figure 5-3. A user makes a write, followed by a read from a stale replica. To prevent this anomaly, we need read-after-write consistency.

Problems with Replication Lag

- Eventual Consistency
  - *Replication lag* can be seconds or minutes
- Reading Your Own Writes: Some Possible Techniques:
  - When reading something that the user may have modified, read it from the leader; otherwise, read it from a follower
  - The client can remember the timestamp of its most recent write.
    - The system can ensure that replicas can delegate reads if they aren't up to date at least to that timestamp.

Problems with Replication Lag

- Eventual Consistency
  - *Replication lag* can be seconds or minutes
- Monotonic Reads: Figure 5-4
  - User makes two reads, the second is from a more lagged replica.
  - This will look to the user like information went backward in time.
  - Consistently reading from same replica will solve this.

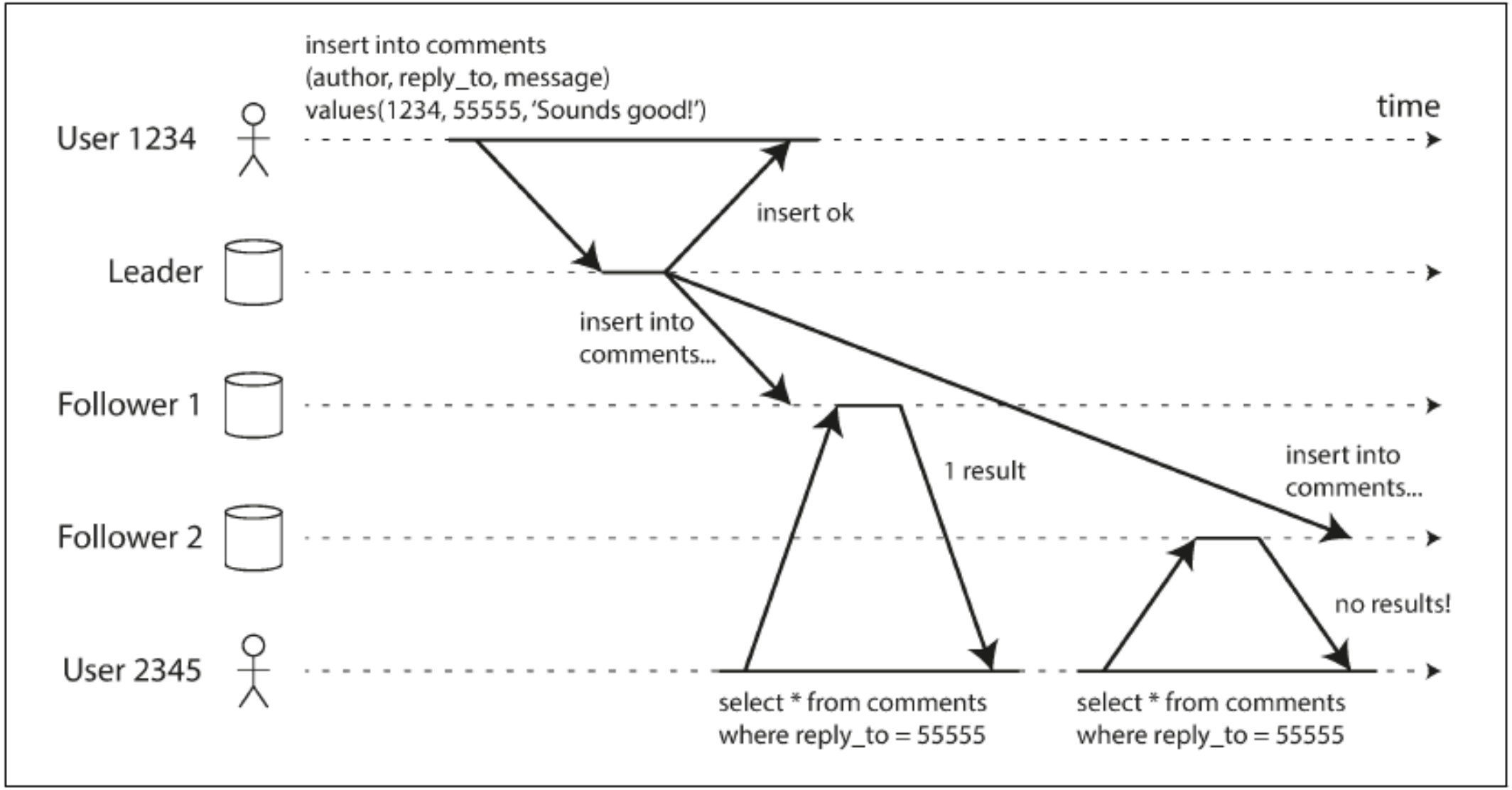


Figure 5-4. A user first reads from a fresh replica, then from a stale replica. Time appears to go backward. To prevent this anomaly, we need monotonic reads.

Problems with Replication Lag

- Eventual Consistency
  - *Replication lag* can be seconds or minutes
- Consistent Prefix Reads
  - Mr. Poons: “How far into the future can you see, Mrs. Cake?”
  - Mrs. Cake: “About ten seconds usually, Mr. Poons.”
  - There is a causal dependency between those two sentences: Mrs. Cake heard Mr. Poons’s question and answered it.

Problems with Replication Lag

- Eventual Consistency
  - *Replication lag* can be seconds or minutes
- Consistent Prefix Reads
  - Mrs. Cake: “About ten seconds usually, Mr. Poons.”
  - Mr. Poons: “How far into the future can you see, Mrs. Cake?”
  - If two writes appear to third person in the reverse order, may seem to violate causality.

Problems with Replication Lag

- Eventual Consistency
  - *Replication lag* can be seconds or minutes
- Consistent Prefix Reads
  - Preventing this kind of anomaly requires another type of guarantee: consistent prefix reads
  - Guarantee: If a sequence of writes happens in a certain order, then anyone reading those writes will see them appear in the same order.
  - Consistently reading from same replica will solve this.

Solutions for Replication Lag

- "How the application behaves if the replication lag increases to several minutes or even hours?"
  - If the answer is "no problem," that's great ☺
  - If the result is a bad experience for users, it's important to design the system to provide a stronger guarantee, such as read-after-write.

Solutions for Replication Lag

- Dealing with these issues in application code is complex and easy to get wrong.
- It would be better if application developers didn't have to worry about subtle replication issues and could just trust their databases to "do the right thing."
- This is why transactions exist: they are a way for a database to provide stronger guarantees so that the application can be simpler.

Solutions for Replication Lag

- Single node transactions have existed for a long time.
- In the move to distributed (replicated and partitioned) databases, many systems:
  - have abandoned them
  - claimed that transactions are too expensive in terms of
    - performance
    - availability
  - asserted that eventual consistency is inevitable in a scalable system
- Next lectures!

Multi-Leader Replication

- Also known as *master–master* or *active/active replication*
- More than one node to accept writes
- Replication still happens in the same way:
  - Each node that processes a write must forward that data change to all the other nodes.
- Each leader also acts as a follower to the other leaders.

Use Cases for Multi-Leader Replication

- Multi-datacenter operation

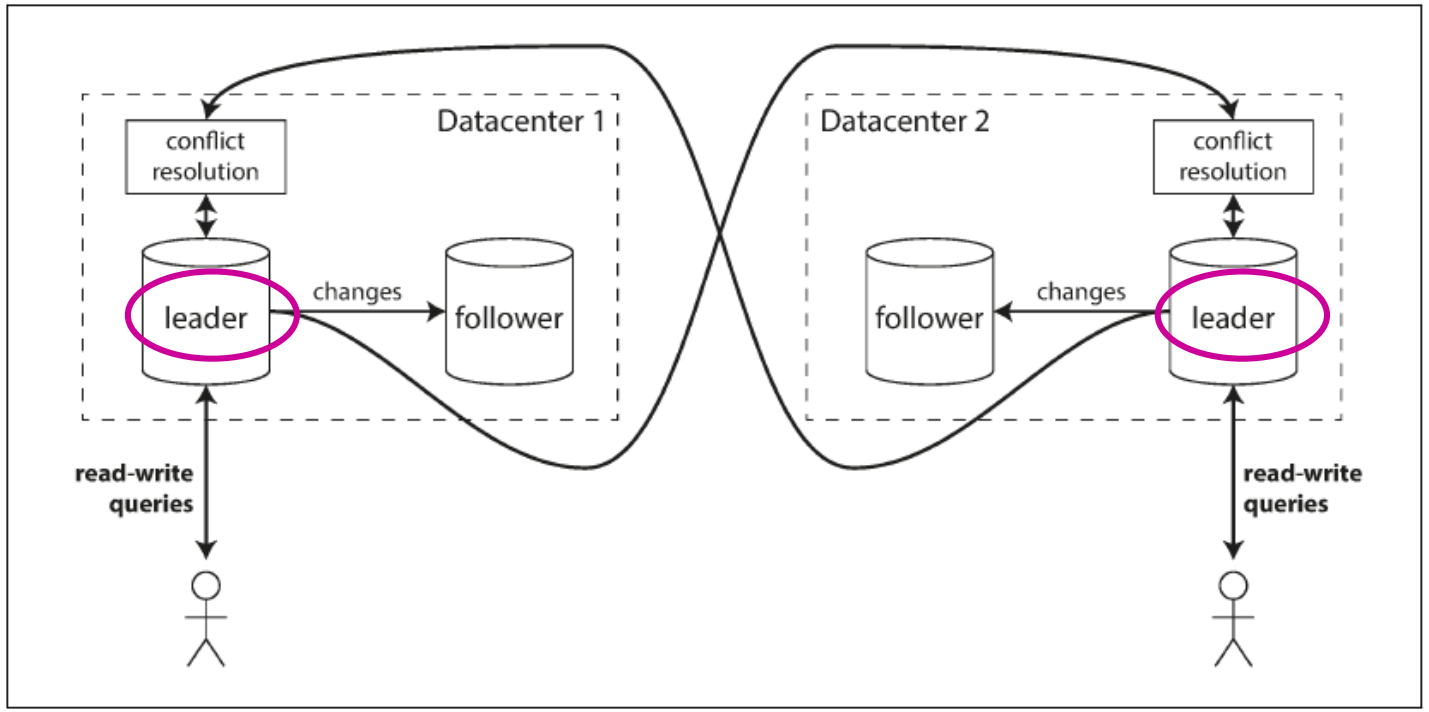


Figure 5-6. Multi-leader replication across multiple datacenters.

Use Cases for Multi-Leader Replication

- Multi-datacenter operation
  - Improves performance
  - Tolerates outages and slow traffic between data centers better

Use Cases for Multi-Leader Replication

- Clients with offline operation
  - Ex: The calendar apps on your mobile phone, your laptop, and other devices
    - At any time, regardless of whether your device currently has an internet connection
      - Able to see your meetings (make read requests)
      - Able to enter new meetings (make write requests) at any time
    - If you make any changes while you are offline, they need to be synced with a server and your other devices when the device is next online

Use Cases for Multi-Leader Replication

- Clients with offline operation
  - Ex: The calendar apps on your mobile phone, your laptop, and other devices
    - Every device has a local database that acts as a leader
    - An asynchronous multi-leader replication process (sync) between the replicas of your calendar on all of your devices.
    - Setup is essentially the same as multi-leader replication between datacenters
  - There are tools that aim to make this kind of multi-leader configuration easier.
    - Ex: **CouchDB** is designed for this mode of operation

CouchDB

Handling Write Conflicts

- The biggest problem with multi-leader replication: write conflicts

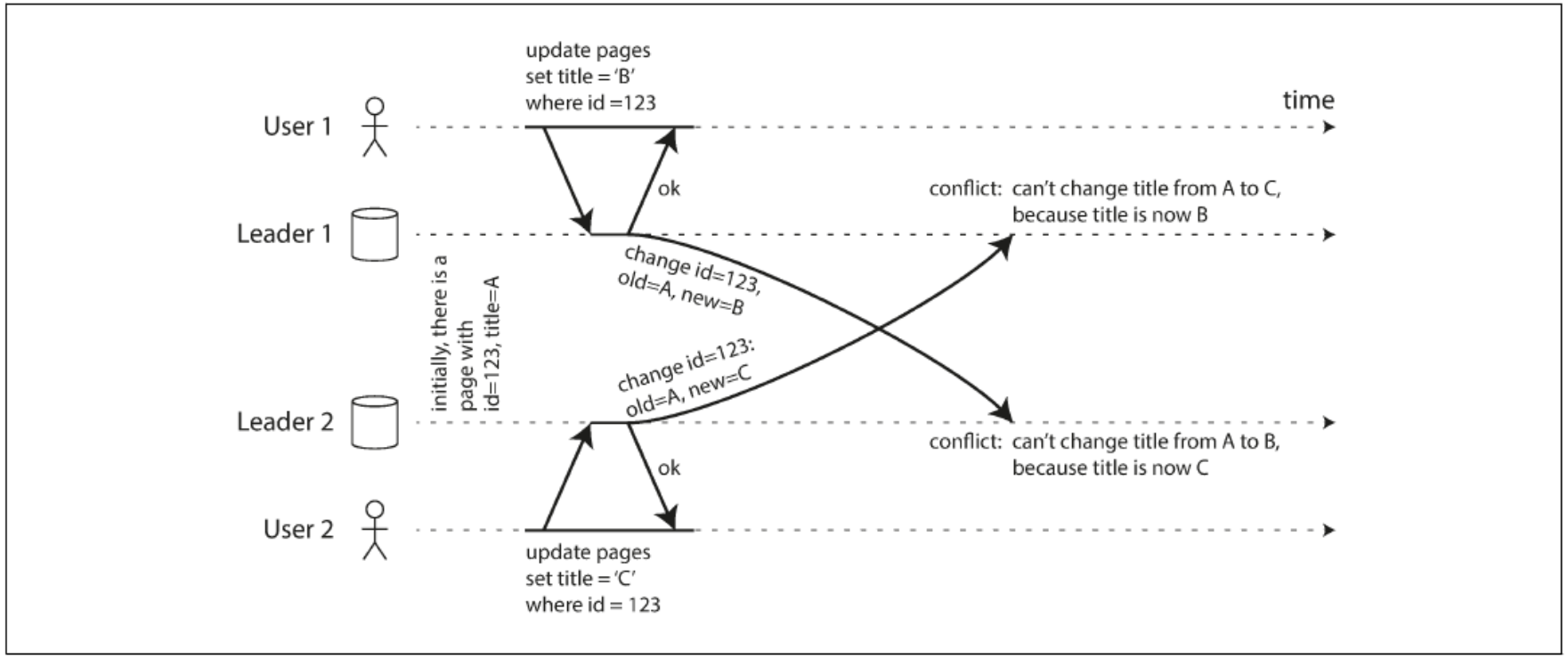


Figure 5-7. A write conflict caused by two leaders concurrently updating the same record.

Handling Write Conflicts

- The biggest problem with multi-leader replication: write conflicts
  - Conflict resolution is required.
- This problem does not occur in a single-leader database.

Handling Write Conflicts

- Synchronous versus asynchronous conflict detection
  - You could make the conflict detection synchronous.
    - Wait for the write to be replicated to all replicas before telling the user that the write was successful.
    - Lose the main advantage of multi-leader replication - allowing each replica to accept writes independently.
  - Conflicts are generally detected asynchronously, so too late to prompt user.

Handling Write Conflicts

- Conflict avoidance
  - Simplest strategy:
    - All writes for a particular record must go through the same leader (ensured by the application)
      - Conflicts cannot occur
    - Requests from a particular user are always routed to the same datacenter and use the leader in that datacenter for reading and writing
    - Different users may have different “home” datacenter

Handling Write Conflicts

- Conflict avoidance
  - However, sometimes you might want to change the designated leader for a record
    - Ex: Perhaps because one datacenter has failed and you need to reroute traffic to another datacenter
    - Ex: Perhaps because a user has moved to a different location and is now closer to a different datacenter
    - In this situation, conflict avoidance breaks down:
      - You have to deal with the possibility of concurrent writes on different leaders.

Handling Write Conflicts

- Converging toward a consistent state
  - Single leader – writes are sequential
  - Multi leader – order of writes can be different at different nodes (no order)
    - Last write wins (LWW) - often based on unique ID (write ID), timestamp
    - Pre-define arbitrary replica precedence rules, then node *A* always overrules *B*
    - Somehow merge the conflicting values being written, e.g. concatenation
      - In Figure 5-7, the merged title might be something like “B/C”
    - Record in a conflict log and resolve later in the application code, possibly with user input

Handling Write Conflicts

- Custom conflict resolution logic
  - Most appropriate way of resolving a conflict may depend on the application
    - Write your own application - specific logic
  - That code may be executed on write or on read:
    - Resolve on write:
      - As soon as the database system detects a conflict in the log of replicated changes, it calls the conflict handler.
    - Resolve on read:
      - When a conflict is detected – all conflicting writes are stored – when next time data is read – multiple versions are returned to the application and application resolves conflicts and writes back to DB.

Single-Leader vs Multi-Leader Replication

Single-leader replication

- Clients send all writes to a single node - the leader
- The leader sends a stream of data change events to the other replicas - followers
- Reads can be performed on any replica, but reads from followers might be stale

Multi-leader replication

- Clients send each write to one of several leader nodes, any of which can accept writes
- The leaders send streams of data change events to each other and to any follower nodes

Single-Leader vs Multi-Leader Replication

Single-leader replication

- Popular
- Fairly easy to understand
- There is no conflict resolution to worry about

Multi-leader replication

- Can be more robust in the presence of
  - faulty nodes
  - network interruptions
  - latency spikes
- Harder to reason about
- Provides only very weak consistency guarantees

Leaderless Replication

- Amazon used leaderless replication for its in-house Dynamo system.
- Open source datastores with leaderless replication models inspired by Dynamo:
  - Riak
  - Cassandra
  - Voldemort
- Either the client directly sends writes to multiple replicas, or a coordinator handles that.

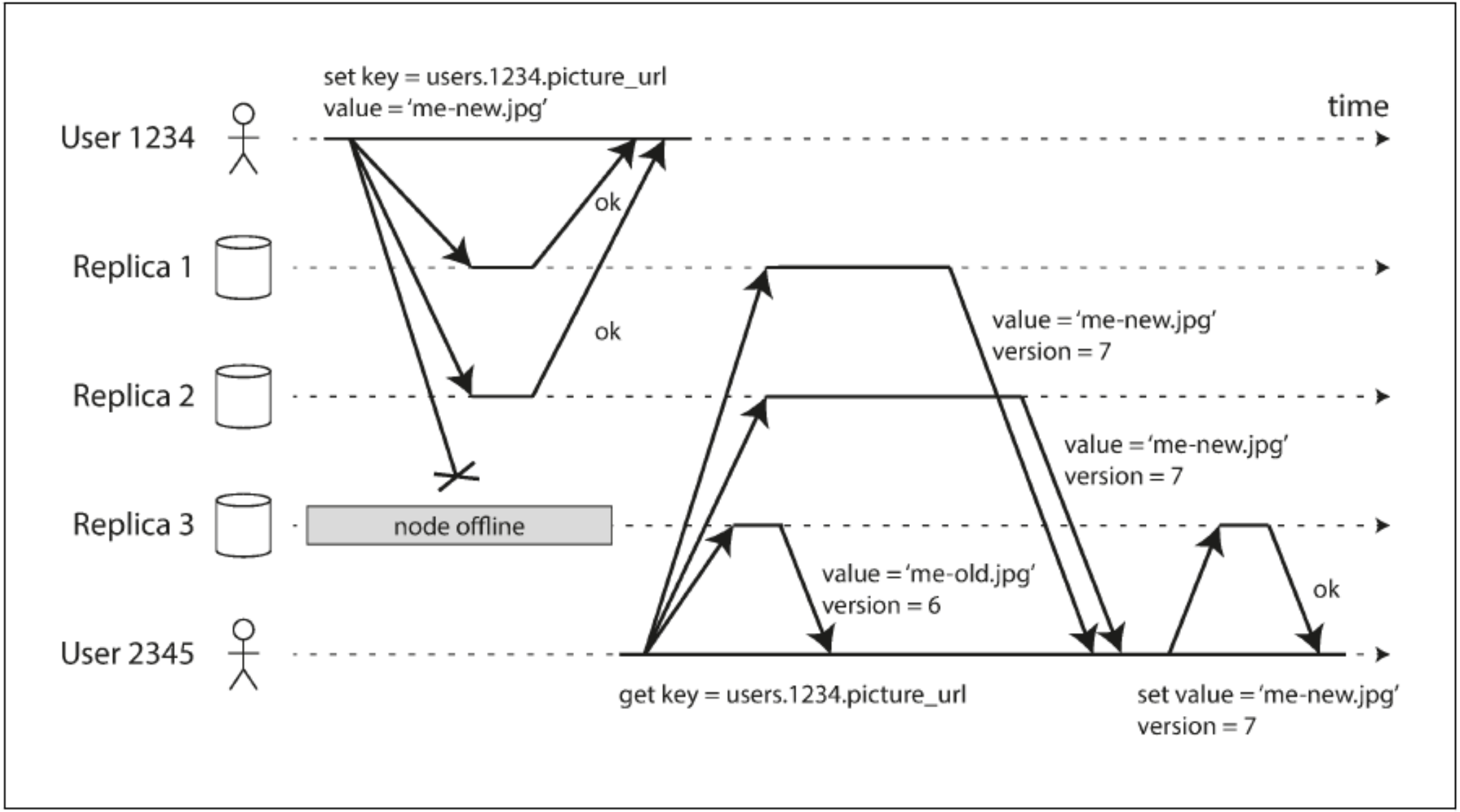


Figure 5-10. A quorum write, quorum read, and read repair after a node outage.

Writing to the Database When a Node Is Down

- When the leader fails in single-leader or multi-leader replication
  - Failover
- In a leaderless configuration, failover does not exist
- Figure 5-10
- The benefit of leaderless is that as long as sufficient nodes complete a write, you can successfully continue.
- When nodes come back up, some of the data is stale.
  - Read requests are also sent to several nodes in parallel.
- Clients detect stale reads by reading from multiple nodes too.

Writing to the Database When a Node Is Down

- Catching up on stale data from missed writes consistency:
  - **Read repair:**
    - When a client detects a stale read, it writes the newer value back.
    - This approach works well for values that are frequently read.
  - **Anti-entropy process:**
    - Background process that constantly looks for differences in data between replicas and copies any missing data from one replica to another.

Quorums for Reading and Writing

- Figure 5-10: We considered the write to be successful even though it was only processed on two out of three replicas.
- If there are *n* nodes, *r* are required for reads, *w* are required for writes, *then* $w + r > n$
- In Figure 5-10: *n*: 3, *w*: 2, *r*: 2
- As long as $w + r > n$, we expect to get an up-to-date value when reading.
  - Because at least one of the *r* nodes we're reading from must be up to date.
- Reads and writes that obey these *r* and *w* values are called *quorum* reads and writes.

Quorums for Reading and Writing

- The quorum condition, $w + r > n$, allows the system to tolerate unavailable nodes:
  - With n = 5, w = 3, r = 3 we can tolerate two unavailable nodes.
    - This case is illustrated in Figure 5-11.

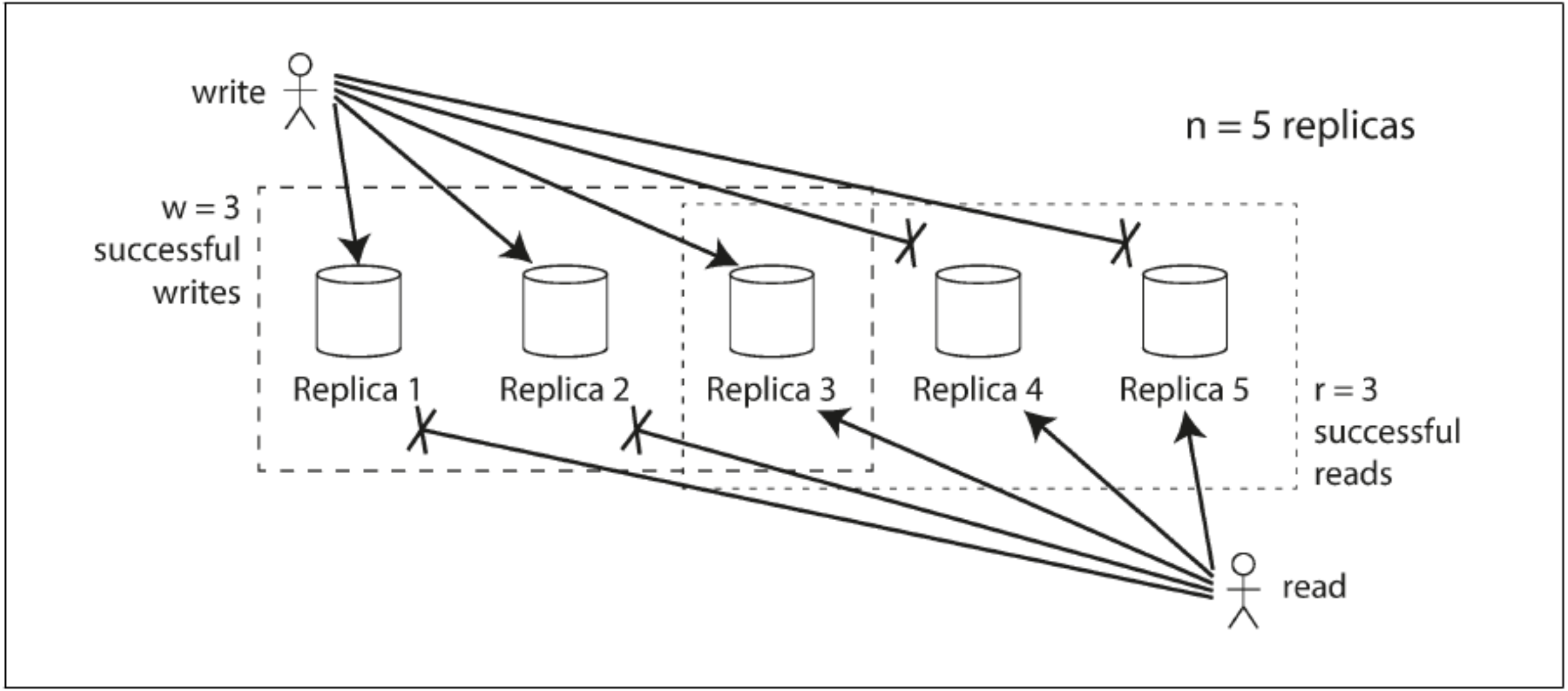


Figure 5-11. If w + r > n, at least one of the r replicas you read from must have seen the most recent successful write.

Quorums for Reading and Writing

- In Dynamo-style databases, the parameters n, w, and r are typically configurable.
- A common choice is to make n an odd number (typically 3 or 5) and to set $w = r = (n + 1) / 2$ (rounded up).

Quorums for Reading and Writing

- Normally, reads and writes are always sent to all *n* replicas in parallel.
- The parameters *w* and *r* determine how many nodes we wait for
  - i.e., how many of the *n* nodes need to report success before we consider the read or write to be successful
- If fewer than the required *w* or *r* nodes are available, writes or reads return an error.

Monitoring Staleness

- With leader-based replication it is usually fairly easy to monitor staleness:
  - You can compare the “version” of each follower relative to each leader to see how far behind the follower is on applying writes.
- Beware that with leaderless replication it’s harder to measure because writes aren’t applied in the same order.
- Further, if anti-entropy background processes aren’t used, it is reasonable behavior for an infrequently read item to be stale even if the write happened forever ago.

Concurrent Writes

- A simple approach is last write wins (LWW).
  - Biggest value of some “timestamp” with each write wins.
- To better address concurrent writes, we need to store more information, and we need to define concurrency.
  - If an operation *A* was aware of operation *B*, we say *B* happened before *A*.
  - If *B* was aware of *A*, then *A* happened before *B*.
  - In all other situations where neither *A* nor *B* were aware of the other, regardless of time according to wall clocks, we define *A* and *B* as concurrent writes.

Summary

- Replication can serve many purposes:
  - High availability
    - Keeping the system running, even when one machine (or several machines, or an entire datacenter) goes down
  - Disconnected operation
    - Allowing an application to continue working when there is a network interruption
  - Latency
    - Placing data geographically close to users, so that users can interact with it faster
  - Scalability
    - Being able to handle a higher volume of reads than a single machine could handle,
    - by performing reads on replicas

Summary

- Three main approaches to replication:
  - Single-leader replication
  - Multi-leader replication
  - Leaderless replication
    - Clients send each write to several nodes, and read from several nodes in parallel in order to detect and correct nodes with stale data.

Summary

- Each approach has advantages and disadvantages.
- Single-leader replication is popular
  - Fairly easy to understand
  - No conflict resolution to worry about
- Multi-leader and leaderless replication can be more robust in the presence of faulty nodes, network interruptions, and latency spikes
  - At the cost of being harder to reason about and providing only very weak consistency guarantees

Synchronous Versus Asynchronous Replication

- Replication can be synchronous or asynchronous
  - Has a profound effect on the system behavior when there is a fault
  - Asynchronous replication can be fast when the system is running smoothly
    - Important to figure out what happens when replication lag increases, and servers fail
    - If a leader fails and you promote an asynchronously updated follower to be the new leader, recently committed data may be lost

Summary

- Replication lag can cause some strange effects
- A few *consistency models* which are helpful for deciding how an application should behave under replication lag:
  - *Read-after-write consistency*
    - Users should always see data that they submitted themselves.
  - *Monotonic reads*
    - After users have seen the data at one point in time, they shouldn't later see the data from some earlier point in time.
  - *Consistent prefix reads*
    - Users should see the data in a state that makes causal sense: for example, seeing a question and its reply in the correct order.

References

- M. Kleppman. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media, Inc., 2017.